DOCUMENTO TÉCNICO

CartaPnk

Arquitectura, seguridad y operación

Frontend y backend separados · Node.js · PostgreSQL · Docker · AWS · Cloudflare

Dato	Detalle
Sistema	CartaPnk — plataforma de menú digital
Sitio (implementación de referencia)	https://cartapnk.xyz
Repositorio	https://github.com/Alkr4/CartaPnk
Versión del documento	1.0 — 10 de noviembre de 2026
Clasificación	Documento técnico — uso interno
Elaborado por	
Destinatario	

Índice

1. Resumen ejecutivo 4
1.1 Alcance y audiencia 4
1.2 Alcance funcional y estado 4
2. Arquitectura 5
2.1 Flujo de una petición 5
2.2 Componentes 5
2.3 Estructura del repositorio 6
3. Backend (API) 6
3.1 Arquitectura MVC 6
3.2 API REST 6
3.3 Modelo de datos 7
3.4 Panel de administración 7
4. Frontend 8
4.1 Prevención de XSS en el cliente 8
4.2 Nginx 8
5. Seguridad 9
5.1 Controles por categoría OWASP Top 10 (2021) 9
5.2 Controles del panel y de las imágenes 9
5.3 Contenedores y red 10
6. Protección del panel de administración 10
6.1 Valor de las capas de Cloudflare 11
6.2 Criterio de selección 11
6.3 Procedimiento de activación 11
7. Ambientes y contenedores 12
7.1 Puesta en marcha 12
7.2 Variables de entorno principales 13
8. Flujo de trabajo, integración continua y CI/CD 13
8.1 Ramas y ambientes 13
8.2 Integración continua (GitHub Actions) 13
8.3 Pipeline de CI/CD con Jenkins (OWASP y MITRE) 14
9. Despliegue en AWS 14
9.1 Recursos 14
9.2 Comandos de scripts/desplegar.sh 15
9.3 Despliegue inicial 15
9.4 Verificación del Auto Scaling 18
9.5 Actualización de la aplicación 18
10. Dominio, HTTPS y Cloudflare 18

10.1 Configuración inicial (una sola vez) 18
10.2 Pasos por cada cuenta de AWS 19
10.3 Configuración que aplica el comando cloudflare 20
10.4 Token de la API de Cloudflare 20
10.5 Diagnóstico 21
11. Operación y recuperación 21
11.1 Escenarios de pérdida de recursos 21
11.2 Reinstalación completa (~25 minutos) 22
11.3 Gestión de secretos 22
11.4 Solución de problemas 22
12. Pruebas y calidad 23
13. Consideraciones para producción y hoja de ruta 23

1. Resumen ejecutivo

CartaPnk es una plataforma de menú digital para restaurantes. Los clientes consultan la carta (destacados, cartas, categorías y platillos con precio e imagen) y arman su pedido desde el navegador; el personal autorizado administra todo el contenido desde un panel protegido.

La solución separa **frontend** y **backend**: un sitio web estático (HTML, CSS y JavaScript sin frameworks) servido por **Nginx**, y una **API REST** en **Node.js + Express** con arquitectura MVC sobre **PostgreSQL**. Se ejecuta en contenedores **Docker** y se despliega en **AWS** con alta disponibilidad (balanceador de carga y Auto Scaling), base de datos administrada (RDS), almacenamiento de imágenes en S3, dominio propio y HTTPS de extremo a extremo con Cloudflare.

Elemento	Detalle
Sitio público (implementación de referencia)	https://cartapnk.xyz
Panel de administración	`/admin` del sitio (acceso solo para el rol administrador; el sitio público no expone ningún formulario de acceso)
Repositorio	https://github.com/Alkr4/CartaPnk (ramas `main` = producción, `develop` = QA)
Frontend	HTML, CSS y JavaScript sin frameworks; Nginx (imagen `nginx-unprivileged`)
Backend	Node.js 24 LTS, Express 5, API JSON bajo `/api`, arquitectura MVC
Base de datos	PostgreSQL 17 (Amazon RDS en producción)
Imágenes	Amazon S3 (bucket privado) o disco local según el ambiente
Infraestructura	AWS (VPC, RDS, Application Load Balancer, Auto Scaling, S3, ECR, ACM) definida como código con CloudFormation; Cloudflare como DNS, CDN y WAF
Calidad	136 pruebas automatizadas del backend (Jest + Supertest), 16 del frontend (node:test + jsdom), análisis estático con ESLint y eslint-plugin-security

1.1 Alcance y audiencia

Este documento describe la arquitectura, los controles de seguridad, los ambientes, el despliegue y la operación de la plataforma. Está dirigido a los equipos de desarrollo, operaciones (DevOps) y seguridad que mantendrán el sistema.

1.2 Alcance funcional y estado

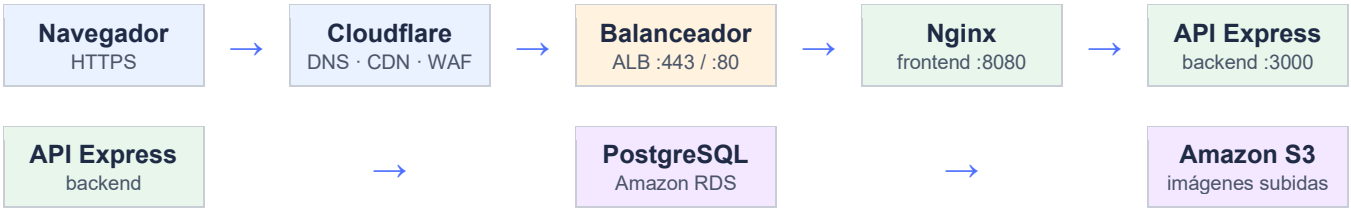
Componente	Estado
Carta digital pública y carrito de pedido	**Implementado** y en producción
Panel de administración (CRUD de restaurantes, cartas, categorías y platillos; imágenes; moderación de comentarios)	**Implementado** y en producción
Tres ambientes (desarrollo, QA, producción) con Docker y Compose	**Implementado**
Infraestructura AWS: RDS, balanceador, Auto Scaling, S3, ECR	**Implementado** y en producción
Dominio propio con HTTPS de extremo a extremo (ACM + Cloudflare *Full strict*)	**Implementado** y en producción

Componente	Estado
Automatización del despliegue y de la configuración de Cloudflare (script y API)	**Implementado**
Integración continua (GitHub Actions: calidad, frontend, infraestructura)	**Implementado**
Endurecimiento adicional del panel en Cloudflare (WAF, rate limiting, Access)	Recomendado; ver sección 6
Pipeline de CI/CD con Jenkins y controles OWASP / MITRE	Diseñado, con `Jenkinsfile` de referencia; ver sección 8.3

2. Arquitectura

2.1 Flujo de una petición

El navegador se comunica **únicamente con Nginx** (mismo origen). Nginx entrega las páginas estáticas y reenvía `/api`, `/media` y `/salud` a la API, que no publica ningún puerto. Al operar bajo un único origen no se requiere CORS y la cookie de sesión conserva los atributos `HttpOnly` y `SameSite=Lax`.



2.2 Componentes

Componente	Responsabilidad	Tecnología
Frontend (`frontend/`)	Páginas y lógica del navegador: carta, carrito y panel de administración. Cabeceras de seguridad (CSP) y proxy hacia la API.	HTML + CSS + JavaScript, Bootstrap 5, jQuery y Owl Carousel; Nginx `nginx-unprivileged`
Backend (`backend/`)	API REST: datos, sesión, autenticación y autorización por rol, validación, subida de imágenes.	Node.js 24, Express 5, `pg`, `express-session`, `helmet`, `multer`, `bcryptjs`
Base de datos	Datos de negocio, sesiones y contadores del límite de intentos de acceso (la aplicación no guarda estado local).	PostgreSQL 17
Almacén de imágenes	Fotos de restaurantes y platillos subidas desde el panel.	Amazon S3 (privado) o disco local
Infraestructura (`infra/`)	Red, base de datos, balanceador, Auto Scaling y bucket como código.	AWS CloudFormation
Automatización (`scripts/desplegar.sh`)	Construcción, publicación, despliegue, configuración del dominio y destrucción del entorno.	Bash, AWS CLI y API de Cloudflare

2.3 Estructura del repositorio

```
backend/    API (Express, MVC): src/  db/  tests/  tools/  Dockerfile
frontend/   Sitio web: public/ (HTML, CSS, JS, imágenes)  nginx/  tests/  Dockerfile
infra/      Plantilla de CloudFormation (VPC, RDS, ALB, Auto Scaling, S3)
scripts/    desplegar.sh (despliegue y operación en AWS)
docs/       Documentación técnica
env/        Plantillas de configuración por ambiente (*.env.example)
docker-compose*.yml   Los tres ambientes        .github/workflows/ci.yml   Integración continua
```

3. Backend (API)

3.1 Arquitectura MVC

Cada petición atraviesa `routes` → `middlewares` → `controllers` → `models` → PostgreSQL y vuelve como JSON; la capa de presentación corresponde al frontend.

Carpeta (`backend/src/`)	Responsabilidad
`models/`	Acceso a datos. **Es el único lugar con SQL** y siempre con parámetros (`$1, $2…`). Usa borrado lógico (`eliminado`) y transacciones cuando se escriben varias tablas.
`controllers/`	Casos de uso: validan la entrada, llaman a los modelos y responden JSON.
`routes/`	Mapa URL → controlador bajo `/api`, más `/media` (imágenes) y `/salud`.
`middlewares/`	Seguridad transversal: cabeceras, sesión, CSRF, límites de intentos, autorización por rol y manejo de errores.
`storage/`	Imágenes subidas: validación por firma y almacén local o S3.
`config/`	Configuración por ambiente leída de variables de entorno (sin secretos en el código).
`utils/`	Validación por esquemas (lista blanca de campos), formato y URL de fotos.

3.2 API REST

Todas las respuestas son JSON. Los errores tienen la forma `{ "error": "mensaje" }`; los de validación (HTTP 422) añaden `errores: { campo: mensaje }`. Las peticiones que modifican datos exigen el token CSRF en la cabecera `X-CSRF-Token`.

Endpoints públicos

Método y ruta	Descripción
`GET /api/sesion`	Estado de la sesión (usuario o nulo) y token CSRF.
`GET /api/restaurantes/primero`	Id del primer restaurante activo (la página de inicio muestra ese).
`GET /api/restaurantes/:id`	Carta completa: restaurante, destacados, cartas > categorías > platillos y comentarios.
`GET /api/carrito`	Productos de la sesión y datos del restaurante.
`POST /api/carrito/items`	Agrega un platillo por id. El precio y el nombre se obtienen de la base de datos, nunca del cliente.

Método y ruta	Descripción
`DELETE /api/carrito/items/:posicion` · `DELETE /api/carrito`	Quita un producto o vacía el pedido.
`GET /media/…` · `GET /salud`	Imágenes subidas (solo claves con formato exacto) · estado de salud para el balanceador (consulta la base de datos).

Endpoints de administración (rol admin)

Método y ruta	Descripción
`POST /api/admin/login` · `POST /api/admin/logout`	Acceso y cierre de sesión. El acceso responde igual ante usuario inexistente, contraseña incorrecta o rol distinto de admin.
`/api/admin/restaurantes` (GET, POST) y `/restaurantes/:id` (GET, PUT, DELETE)	CRUD de restaurantes con su dirección e imagen.
`/api/admin/restaurantes/:id/cartas`, `/cartas/:id`	CRUD de cartas.
`/api/admin/cartas/:id/categorias`, `/categorias/:id`	CRUD de categorías.
`/api/admin/categorias/:id/items`, `/items/:id`	CRUD de platillos (precio, descripción, puntuación, destacado, imagen).
`DELETE /api/admin/comentarios/:id`	Moderación de comentarios.

3.3 Modelo de datos

Tabla	Contenido
`direcciones`, **`restaurantes`**	Datos del restaurante (RUT, nombre, contacto, foto) y su dirección.
`cartas`, **`categorias`**, **`items`**	Jerarquía del menú. `items` guarda precio (entero), puntuación 0–5, orden, visible, destacado y foto.
`usuarios`	Correo, hash bcrypt (`password_hash`), `rol` (`admin` o `cliente`) y `activo`.
`comentarios`	Comentarios publicados (solo lectura en el sitio; el administrador los modera).
`sesiones`	Sesiones de servidor (connect-pg-simple).
`limite_intentos`	Contadores del límite de intentos de acceso, compartidos entre todas las instancias.

Las migraciones (`backend/db/migrations/`) se aplican automáticamente al iniciar el contenedor, con un bloqueo de PostgreSQL que evita conflictos cuando varias instancias arrancan a la vez. El catálogo inicial se carga una sola vez si la base está vacía.

3.4 Panel de administración

El administrador ingresa en `/admin`. Puede crear, editar, ocultar y eliminar restaurantes, cartas, categorías y platillos, subir imágenes y moderar comentarios. "Eliminar" es **borrado lógico**: el dato deja de mostrarse pero se conserva en la base. "Ocultar" lo retira del sitio y lo mantiene visible en el panel.

El primer administrador se crea al iniciar con las variables `ADMIN_EMAIL` y `ADMIN_PASSWORD` (mínimo 12 caracteres). Los usuarios de demostración existen únicamente en los ambientes de desarrollo y QA.

> **Alcance funcional**
> El acceso es exclusivo del personal autorizado; los clientes no inician sesión y, por lo tanto, no publican comentarios. Los comentarios existentes se muestran en modo lectura y son moderados desde el panel.

4. Frontend

Sitio estático sin frameworks de JavaScript. Cada pantalla es un documento HTML con su propio script:

Archivo (`frontend/public/`)	Pantalla y ruta
`carta.html` + `js/carta.js`	Carta digital: `/` y `/restaurantes/:id` (destacados, pestañas por carta, comentarios).
`carrito.html` + `js/carrito.js`	Productos seleccionados: `/carrito`.
`admin.html` + `js/admin.js`	Panel de administración: `/admin`. Aplicación de una sola página; las pantallas usan `#/…`.
`js/comun.js`	Cliente de la API (gestiona el token CSRF), creación segura de elementos, formato de moneda e iconos.
`404.html`	Página de error para rutas inexistentes.

4.1 Prevención de XSS en el cliente

- Todo dato proveniente de la API se inserta como **nodo de texto** (función `h()`), nunca como HTML: un nombre o comentario que contenga `<script>` se muestra literalmente y no se ejecuta.
- Las URL de imágenes se aceptan solo si son rutas internas (`/media/…`, `/imagenes/…`); valores como `javascript:` o `//otro-sitio` se reemplazan por la imagen por defecto.
- Las páginas no contienen scripts en línea ni manejadores `onclick`, por lo que la política `script-src 'self'` puede ser estricta.
- La única excepción de `innerHTML` son los iconos SVG, que son constantes del propio archivo.

4.2 Nginx

Ruta	Comportamiento
`/`, `/restaurantes/<id>`, `/carrito`, `/admin`	Entrega la página estática correspondiente. El panel se sirve con `no-store` y `noindex`.
`/css`, `/js`	Se revalidan siempre (ETag): un despliegue nuevo se refleja de inmediato.
`/img`, `/imagenes`, `/vendors`	Caché de 7 días.
`/api/`, `/media/`, `/salud`	Se reenvían a la API (`back:3000`) con `X-Forwarded-For` y `X-Forwarded-Proto`.
Cualquier otra ruta	Respuesta 404 con la página de error.

Las páginas incluyen Content-Security-Policy (`default-src 'self'`, `script-src 'self'`, `frame-ancestors 'none'`…), `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` y `Permissions-Policy`. La imagen usa `nginx-unprivileged`: el proceso se ejecuta sin privilegios en el puerto 8080.

5. Seguridad

5.1 Controles por categoría OWASP Top 10 (2021)

Categoría	Controles implementados
A01 · Control de acceso	Autorización por rol (`requiereAdmin`) comprobada en el servidor en cada petición (401 sin sesión, 403 con otro rol). Token CSRF por sesión comparado en tiempo constante. Consultas filtradas por `eliminado IS NULL`.
A02 · Fallas criptográficas	Contraseñas con bcrypt (12 rondas). HTTPS de extremo a extremo (Cloudflare → ALB con certificado de ACM). TLS verificado hacia RDS. Cifrado en reposo en RDS y S3 (AES-256). Cookie `Secure` en producción y HSTS.
A03 · Inyección	Solo consultas parametrizadas. Identificadores de la URL validados como enteros positivos. Entrada validada con esquemas de lista blanca (sin asignación masiva). En el cliente, los datos se insertan como texto (sin XSS).
A04 · Diseño inseguro	Defensa en profundidad (sección 6). El precio y el nombre del carrito provienen de la base de datos. Aplicación sin estado: sesiones, carrito y límites en PostgreSQL. API sin puertos publicados.
A05 · Configuración incorrecta	`helmet` y CSP; sin `X-Powered-By`; manejador de errores que no expone detalles internos. Contenedores de solo lectura, `cap_drop: ALL` y `no-new-privileges`. S3 con bloqueo de acceso público. Secretos solo por variables de entorno.
A06 · Componentes vulnerables	`npm audit` de producción en cada integración continua, `eslint-plugin-security`, versiones actuales de dependencias. Escaneo de imágenes y dependencias previsto en el pipeline de Jenkins (Trivy, Dependency-Check).
A07 · Identificación y autenticación	Mensaje genérico ante cualquier fallo de acceso, igualación de tiempos de respuesta, límite de 5 intentos fallidos por IP y correo en 15 minutos (contador global en PostgreSQL, HTTP 429), sesión nueva y token CSRF nuevo al autenticar, sesión de administrador de 2 horas, cookie `HttpOnly` y `SameSite=Lax`.
A08 · Integridad de software y datos	Integración continua con chequeos obligatorios, imágenes versionadas por etiqueta, validación del tipo real de las imágenes por firma, migraciones con control de versión y bloqueo.
A09 · Registro y monitoreo	Registros estructurados en JSON con rotación, endpoint `/salud` consultado por el balanceador cada 15 segundos y reemplazo automático de instancias no saludables.
A10 · SSRF	No aplica: la API no realiza peticiones a URL suministradas por el usuario.

5.2 Controles del panel y de las imágenes

Control	Cómo funciona
Autorización por rol	El frontend solo muestra u oculta pantallas; toda decisión de acceso se toma en el servidor.
Validación de entrada	Esquemas con lista blanca de campos (tipo, largo y rango). Los campos no declarados se ignoran.

Control	Cómo funciona
Subida de imágenes	Tipo real por **firma** (JPG, PNG o WEBP; SVG rechazado), máximo 2 MB, nombre aleatorio de 32 caracteres; nada se escribe antes de validar.
Entrega de imágenes	`X-Content-Type-Options: nosniff` y `Content-Security-Policy: sandbox`; solo se sirven claves con el formato exacto (sin recorrido de rutas).
Respuestas de la API	`Cache-Control: no-store` y `X-Robots-Tag: noindex`: dependen de la sesión y no deben almacenarse en cachés.
Almacén S3	Bucket privado: bloqueo de acceso público, cifrado AES-256, política que exige HTTPS y credenciales por rol de instancia (IMDSv2).

5.3 Contenedores y red

- La API **no publica puertos**: solo el Nginx de la misma instancia la alcanza, por una red interna de Docker.
- Contenedores en producción: sistema de archivos de solo lectura, `cap_drop: ALL`, `no-new-privileges`, límites de memoria y de procesos, registros rotados.
- Base de datos en subredes privadas, sin acceso público, con conexión TLS verificada contra el certificado raíz de Amazon (incluido en la imagen).
- Grupos de seguridad de mínimo privilegio: Internet → balanceador (80/443) → instancia (80, solo desde el balanceador) → RDS (5432, solo desde la instancia).

6. Protección del panel de administración

El panel se protege en **capas independientes**: si una falla, las siguientes siguen en pie. Las cuatro primeras están activas en la aplicación; las tres de Cloudflare constituyen el endurecimiento recomendado.

#	Capa	Dónde	Estado
1	Contraseña con bcrypt, error genérico, tiempos igualados	Aplicación	✓ Activa
2	Rol admin en el servidor, CSRF, sesión de 2 h, cookie HttpOnly + Secure + SameSite	Aplicación	✓ Activa
3	Límite de intentos de acceso (5 por IP y correo en 15 min, en PostgreSQL)	Aplicación	✓ Activa
4	Respuestas con `no-store` y `noindex`	Aplicación / Nginx	✓ Activa
5	Regla WAF: solo orígenes autorizados alcanzan `/admin` y `/api/admin`	Cloudflare	Recomendada
6	Rate limiting en el borde: Cloudflare corta los intentos antes de llegar a AWS	Cloudflare	Recomendada
7	Cloudflare Access: código por correo antes de mostrar el formulario de acceso	Cloudflare	Recomendada

6.1 Valor de las capas de Cloudflare

- **Reducen la superficie de ataque.** Sin ellas, cualquier origen en Internet puede alcanzar el formulario de acceso; con ellas, ni siquiera lo ve. Una vulnerabilidad futura en la aplicación o en una dependencia no sería explotable desde fuera.
- **Cortan el abuso donde es más barato.** El límite de intentos de la aplicación detiene la fuerza bruta, pero la petición ya consumió CPU y base de datos. En el borde, Cloudflare absorbe el tráfico malicioso sin costo de infraestructura (relevante ante denegación de servicio).
- **Añaden un segundo factor independiente de la aplicación.** Con Access se requiere acceso a un correo autorizado además de usuario y contraseña.
- **Aplican el principio de defensa en profundidad** (ISO/IEC 27034, OWASP ASVS V2 y V4): ningún control es infalible, por eso se apilan.

6.2 Criterio de selección

Escenario de acceso	Capa recomendada
Administración desde IP fija (oficina) o a través de VPN corporativa	**Regla WAF por IP (5)**: simple y sin dependencia de terceros.
Administradores remotos o con IP variable	**Cloudflare Access (7)**: autenticación por correo independiente de la IP (gratuito hasta 50 usuarios; requiere activar Zero Trust).
Exposición a tráfico automatizado hostil	**Rate limiting en el borde (6)** sobre `POST /api/admin/login` (el plan gratuito permite una regla).

Las capas 5 a 7 no están activas en el despliegue actual; las capas 1 a 4 cubren el riesgo principal (adivinación de contraseñas, escalamiento de privilegios, CSRF y secuestro de sesión) y cuentan con pruebas automatizadas (`backend/tests/admin-auth.test.js`, `admin-crud.test.js`).

6.3 Procedimiento de activación

Regla WAF por IP

1. Cloudflare → la zona → **Security** → **WAF** → **Custom rules** → **Create rule**.
2. Pulsar **Edit expression** y pegar (reemplazando la IP de ejemplo por la autorizada):

```
((starts_with(http.request.uri.path, "/admin") or starts_with(http.request.uri.path,
"/api/admin"))
 and ip.src ne 203.0.113.10)
```

Acción **Block** y **Deploy**. Deben cubrirse **ambas rutas**: la página (`/admin`) y la API del panel (`/api/admin`); proteger solo la página dejaría la API accesible. Para varias IP: `ip.src notin {203.0.113.10 198.51.100.7}`.

Cloudflare Access

1. Cloudflare → **Zero Trust** (crear el "team name" y elegir el plan Free).
2. **Access** → **Applications** → **Add an application** → **Self-hosted**, dominio `<DOMINIO>`, ruta `admin`; agregar una segunda entrada con la ruta `api/admin`.
3. Política **Allow** → **Emails** con los correos autorizados y método de acceso One-time PIN.

Rate limiting en el borde

Security → WAF → Rate limiting rules: regla sobre `POST /api/admin/login` (por ejemplo, 5 peticiones por minuto por IP, acción Block).

7. Ambientes y contenedores

El mismo código se ejecuta con distinta configuración en cada ambiente. Cada uno usa un nombre de proyecto de Compose propio (`pnk-dev`, `pnk-qa`, `pnk-prod`), por lo que tiene su red y su volumen de datos.

	Desarrollo	QA	Producción
`APP_ENV`	`development`	`qa`	`production`
Imágenes Docker	API con nodemon; frontend con archivos montados	API `produccion` + contenedor de pruebas	API `produccion` + frontend Nginx
Datos	Catálogo + usuarios de demostración	Catálogo + usuarios de demostración	Catálogo; **sin** usuarios de demostración; administrador por variables
Secretos	Valores por defecto permitidos	Obligatorios	Obligatorios (≥ 32 caracteres)
Cookie Secure / HSTS	No	No (HTTP local)	Sí (HTTPS)
Puertos	Sitio 3000, API 127.0.0.1:3100, BD 127.0.0.1:5432	Sitio 3001	Sitio solo en 127.0.0.1 (o 80); **API y BD sin puertos**
Endurecimiento	—	—	Solo lectura, `cap_drop ALL`, límites de memoria y PIDs, logs rotados

7.1 Puesta en marcha

Requisitos: Docker con Compose v2 y Node 22+ (este último solo para el script que genera los archivos `env`).

```
node tools/init-env.js dev qa      # crea env/dev.env y env/qa.env con secretos aleatorios
npm run docker:dev                 # DESARROLLO  → http://localhost:3000
npm run docker:qa                  # QA          → lint + pruebas + auditoría
npm run docker:qa:app              # QA (sitio)  → http://localhost:3001
npm run docker:prod                # PRODUCCIÓN
```

Nota sobre AWS CloudShell

CloudShell incluye Docker pero no el plugin `docker compose`. Se instala una vez por usuario:

```
mkdir -p ~/.docker/cli-plugins && curl -SL
https://github.com/docker/compose/releases/download/v2.29.7/docker-compose-linux-x86_64 -o
~/.docker/cli-plugins/docker-compose && chmod +x ~/.docker/cli-plugins/docker-compose
```

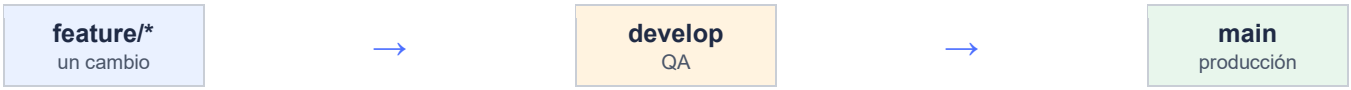
7.2 Variables de entorno principales

Variable	Uso
`APP_ENV`	`development`, `qa` o `production`; determina los controles de arranque.
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`	Conexión a PostgreSQL (`DB_SSL=true` con RDS).
`SESSION_SECRET`	Firma de la cookie de sesión (≥ 32 caracteres en QA y producción).
`COOKIE_SECURE`, `TRUST_PROXY`	Cookie Secure (HTTPS) y número de proxys de confianza para obtener la IP real del visitante. Compose: 1; AWS: 3 (Nginx + balanceador + Cloudflare).
`SEED_CATALOGO`, `SEED_USUARIOS_DEMO`	Carga del catálogo inicial y de usuarios de demostración.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Primer administrador (contraseña ≥ 12 caracteres).
`ADMIN_SESION_HORAS`	Duración de la sesión de administrador (por defecto 2).
`STORAGE_DRIVER`, `S3_BUCKET`, `AWS_REGION`, `UPLOADS_DIR`	Almacén de imágenes: `local` (disco) o `s3`.

8. Flujo de trabajo, integración continua y CI/CD

8.1 Ramas y ambientes

El código reside en un único repositorio. Los ambientes no se transfieren código entre sí: cada uno obtiene (`git pull`) la rama que le corresponde y la ejecuta con su propia configuración.



Rama	Propósito
`feature/<nombre>`	Un cambio concreto. Se integra mediante Pull Request hacia `develop`.
`develop`	Cambios integrados, listos para validarse en QA.
`main`	Código aprobado; es lo que se despliega a producción.

Política recomendada para `main`: protección de rama en GitHub (*Settings* → *Branches*) que exija Pull Request y el éxito de los chequeos `calidad`, `frontend` e `infraestructura`.

8.2 Integración continua (GitHub Actions)

Trabajo	Qué hace
`calidad`	Levanta PostgreSQL, instala el backend y ejecuta lint (ESLint + eslint-plugin-security), las 136 pruebas y `npm audit` de producción.
`frontend`	Ejecuta las 16 pruebas con jsdom y valida la configuración de Nginx construyendo la imagen y ejecutando `nginx -t`.
`infraestructura`	Valida la plantilla de CloudFormation con `cfn-lint` y revisa los scripts con ShellCheck (informativo).

8.3 Pipeline de CI/CD con Jenkins (OWASP y MITRE)

Se propone un pipeline que lleva cada cambio por **calidad** → **seguridad** → **despliegue** y que interrumpe la entrega cuando un control de seguridad detecta hallazgos. El `Jenkinsfile` de referencia está en la raíz del repositorio y apunta a la estructura de contenedores `back` y `front`.

#	Etapa	Herramienta	Qué detecta
1	Secretos	Gitleaks	Claves y contraseñas en el repositorio
2	SAST	Semgrep (OWASP Top 10, Node) + ESLint security	Inyecciones, XSS, uso inseguro de APIs
3	SCA	npm audit + OWASP Dependency-Check	Dependencias con CVE conocidos
4	Calidad	Jest + Supertest en Docker (QA)	Regresiones funcionales y de seguridad
5	Imagen	Trivy	CVE en el sistema base y las librerías
6	DAST	OWASP ZAP (baseline) contra QA	Cabeceras, cookies, XSS/SQLi reflejados
7	MITRE	`backend/tools/mapear-mitre.js`	Consolida los hallazgos por CWE → CAPEC → ATT&CK
8	Aprobación	`input` de Jenkins	Autorización humana para producción
9	Despliegue	SSH + Docker Compose	Prueba de humo `/salud` a través de Cloudflare

MITRE aporta **catálogos** (CVE, CWE, CAPEC, ATT&CK) que las herramientas ya utilizan; el pipeline los consolida en un informe `mitre.md`. OWASP aporta el estándar (Top 10, ASVS) y las herramientas Dependency-Check y ZAP. Jenkins se despliega en un servidor propio (≥ 2 vCPU y 4 GB) con acceso restringido, registro de usuarios desactivado, autorización por roles y credenciales solo en su almacén seguro.

9. Despliegue en AWS

9.1 Recursos

Todo el despliegue es **infraestructura como código** (`infra/pnk-stack.yaml`, CloudFormation) y se opera con un único script. La plantilla no crea recursos IAM: utiliza un perfil de instancia existente, indicado en el parámetro `InstanceProfileName`.

Recurso	Detalle
VPC `10.20.0.0/16`	2 subredes públicas (balanceador e instancias) y 2 privadas (solo RDS); sin NAT Gateway.
RDS PostgreSQL 17	`db.t3.micro`, 20 GB gp3 cifrado, en subredes privadas, sin acceso desde Internet.
Application Load Balancer	Recibe HTTP :80 (y HTTPS :443 con el certificado); comprueba `/salud` cada 15 s y retira del servicio a la instancia que no responde.
Auto Scaling Group	Mínimo 2, máximo 4 instancias `t3.small` en 2 zonas; escala por peticiones por instancia (> 300/min) y por CPU (> 55 %); reemplaza instancias no saludables; actualiza mediante *rolling update*.

Recurso	Detalle
Cada instancia	Dos contenedores en una red interna: `front` (Nginx, publicado en el puerto 80) y `back` (API, sin puertos).
Amazon S3	Bucket privado para las imágenes del panel, compartido por todas las instancias.
Amazon ECR	Repositorio `cartapnk` con dos imágenes: `back-<etiqueta>` y `front-<etiqueta>`.
ACM	Certificado del dominio, validado por DNS.

Las instancias **no guardan estado**: las sesiones, el carrito y los contadores del límite de acceso residen en PostgreSQL y las imágenes en S3, por lo que se pueden añadir o retirar instancias sin afectar a los usuarios.

9.2 Comandos de scripts/desplegar.sh

Comando	Qué hace
`verificar`	Comprueba herramientas y permisos de la cuenta; no crea nada.
`up [--sin-build]`	Genera los secretos, construye las dos imágenes, las publica en ECR y crea o actualiza toda la infraestructura. Espera la salud del balanceador e imprime la URL.
`status` · `url`	Estado de la pila, instancias, salud del balanceador y RDS · URL pública.
`certificado <dominio>`	Solicita el certificado de ACM; con token de Cloudflare crea el registro de validación y espera la emisión.
`dominio [<dominio>]`	Imprime la configuración de Cloudflare correspondiente a la cuenta actual y comprueba si el dominio responde.
`cloudflare [<dominio>] [--reemplazar]`	Crea o corrige los registros DNS y aplica los ajustes de la zona mediante la API de Cloudflare.
`tunel`	Publica la URL con un túnel temporal de Cloudflare (sin dominio propio).
`carga`	Genera tráfico para demostrar el Auto Scaling.
`down [--ecr]`	Destruye el entorno (pide confirmación). Vacía el bucket S3 antes de eliminar la pila.

9.3 Despliegue inicial

Requisitos previos

El despliegue se ejecuta desde cualquier equipo con las herramientas de la tabla. **AWS CloudShell** (icono `>_` en la barra superior de la consola de AWS) las incluye casi todas y no requiere configurar credenciales; es la opción más simple.

Herramienta	Para qué se usa	Cómo verificarla
AWS CLI v2	Operar la cuenta de AWS	`aws --version`
Docker	Construir y publicar las imágenes	`docker info`
git	Clonar el repositorio	`git --version`

Herramienta	Para qué se usa	Cómo verificarla
bash, openssl, curl	Ejecutar el script, generar secretos y comprobar el sitio	`openssl version`

Dónde obtener las credenciales de AWS

El script usa las credenciales que tenga configuradas la AWS CLI. Según el tipo de cuenta se obtienen así:

Tipo de cuenta	Dónde se obtienen	Observaciones
AWS CloudShell	No hay que obtenerlas: la consola de AWS las entrega a la sesión.	Recomendado. Se hereda el usuario con el que se inició sesión en la consola.
Cuenta propia (usuario IAM)	Consola de AWS → **IAM** → **Users** → el usuario → **Security credentials** → **Create access key** → caso de uso *Command Line Interface (CLI)*. Copiar *Access key ID* y *Secret access key*.	El secreto se muestra **una sola vez**. Usar un usuario con permisos acotados, no la cuenta raíz.
Laboratorio (por ejemplo AWS Academy)	Iniciar el laboratorio (**Start Lab**) → **AWS Details** → **AWS CLI** → **Show**. Copiar las tres líneas del bloque.	Incluyen un *session token* y **caducan** al terminar la sesión (≈ 4 h): hay que copiarlas de nuevo.
IAM Identity Center (SSO)	`aws configure sso` y luego `aws sso login`.	Las credenciales se renuevan con cada inicio de sesión.

Para pegar las credenciales manualmente (usuario IAM o laboratorio):

```
mkdir -p ~/.aws && nano ~/.aws/credentials      # pegar el bloque y guardar
aws configure set region us-east-1              # región donde se despliega
aws sts get-caller-identity                     # comprobar que las credenciales funcionan
```

Formato del archivo `~/.aws/credentials` (valores de ejemplo):

```
[default]
aws_access_key_id=AKIAEJEMPLO1234567
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEJEMPLOKEY
aws_session_token=IQoJb3JpZ2luX2VjEJ3...      # solo en credenciales temporales
```

Respuesta esperada de `aws sts get-caller-identity` (la cuenta debe ser la que se desea usar):

```
{
    "UserId": "AIDAEJEMPLO1234567",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/<usuario>"
}
```

> **Permisos y seguridad de las credenciales**
>
> La identidad necesita permisos sobre CloudFormation, EC2 y VPC, RDS, Elastic Load Balancing, Auto Scaling, ECR, S3 y ACM, lectura de parámetros de SSM y poder asignar el perfil de instancia indicado (`iam:PassRole`). El comando `verificar` lo comprueba sin crear nada.
>
> **Nunca** se suben credenciales a Git ni se pegan en chats o tickets. Las claves de acceso permanentes deben rotarse periódicamente.

Pasos

1. **Clonar el repositorio:** `git clone https://github.com/Alkr4/CartaPnk.git && cd CartaPnk`
2. **Verificar la cuenta** (no crea nada): `./scripts/desplegar.sh verificar`. Cada línea debe salir con ✓; si alguna sale con ✗, el mensaje indica qué servicio falta.
3. **Desplegar:** `./scripts/desplegar.sh up`. Genera los secretos, construye y publica las imágenes y crea la infraestructura (~15 minutos; la creación de RDS es lo más lento). No cerrar la terminal.
4. **Comprobar:** `./scripts/desplegar.sh status` y abrir la URL que imprime el script.

Salida esperada de `verificar` (resumida):

```
==> Verificando herramientas y permisos (no se crea nada)
  ✓ herramienta instalada: aws
  ✓ credenciales AWS válidas (cuenta 123456789012, región us-east-1)
  ✓ EC2/VPC: listar VPCs
  ✓ RDS: PostgreSQL en la clase db.t3.micro
  ✓ ELB: Application Load Balancer
  ✓ Auto Scaling: grupos
  ✓ ECR: repositorios
  ✓ S3: listar buckets (imágenes del panel)
  ✓ Todo en orden: puede ejecutar './scripts/desplegar.sh up'
```

Salida esperada al terminar `up` (la URL y el bucket son únicos de cada despliegue):

```
================================================================================
 Aplicación desplegada
   URL (HTTP):    http://<balanceador>.us-east-1.elb.amazonaws.com
   Salud:         http://<balanceador>.us-east-1.elb.amazonaws.com/salud
   Base de datos: RDS PostgreSQL (privada): <instancia>.us-east-1.rds.amazonaws.com
   Panel admin:   http://<balanceador>.us-east-1.elb.amazonaws.com/admin   (imágenes en S3:
<bucket>)
   Administrador: admin@cartapnk.cl
   Contraseña:    en .despliegue/secretos.env
================================================================================
```

Comprobación desde la terminal y lectura de la contraseña del administrador:

```
curl -s "$(./scripts/desplegar.sh url)/salud"       # {"estado":"ok","ambiente":"production"}
cat .despliegue/secretos.env                         # contraseña del administrador y demás
secretos
```

Variables opcionales

Se anteponen al comando para ajustar el tamaño o la configuración. Ejemplo: `PNK_INSTANCE_TYPE=t3.medium PNK_ASG_MAX=6 PNK_ADMIN_EMAIL=admin@mi-empresa.com ./scripts/desplegar.sh up`.

Variable	Efecto	Valor por defecto
`PNK_ADMIN_EMAIL`	Correo del primer administrador	`admin@cartapnk.cl`
`PNK_INSTANCE_TYPE`	Tipo de instancia EC2	`t3.small`
`PNK_ASG_MIN`, `PNK_ASG_DESIRED`, `PNK_ASG_MAX`	Tamaño mínimo, deseado y máximo del grupo de Auto Scaling	2, 2 y 4
`PNK_DB_CLASS`, `PNK_DB_ENGINE_VERSION`	Clase y versión de PostgreSQL en RDS	`db.t3.micro` y 17

Variable	Efecto	Valor por defecto
`PNK_INSTANCE_PROFILE`	Perfil de instancia existente que usarán las instancias	`LabInstanceProfile`
`PNK_STACK`, `PNK_ECR_REPO`	Nombre de la pila de CloudFormation y del repositorio ECR	`cartapnk`
`PNK_IMAGE_TAG`	Etiqueta de las imágenes	hash de git + fecha
`AWS_REGION`	Región de despliegue	la configurada en la AWS CLI

Costos
Mientras exista, el entorno genera costos (2 instancias, RDS y balanceador). Cuando no se necesite, destruirlo con `./scripts/desplegar.sh down`. Con credenciales temporales (por ejemplo, un laboratorio), la sesión caduca y las credenciales deben renovarse.

9.4 Verificación del Auto Scaling

```
# Terminal 1: genera carga sobre la API
./scripts/desplegar.sh carga
# Terminal 2: observa el crecimiento del grupo (reacciona en 3-5 minutos)
./scripts/desplegar.sh status
```

9.5 Actualización de la aplicación

Tras un `git pull` con cambios, `./scripts/desplegar.sh up` construye imágenes nuevas, las publica y CloudFormation **reemplaza las instancias de a una**, sin interrumpir el servicio. Para reutilizar las últimas imágenes sin reconstruir: `up --sin-build`.

10. Dominio, HTTPS y Cloudflare

El dominio se registra en cualquier registrador y su DNS se delega a **Cloudflare** (plan gratuito), que actúa como DNS, CDN y WAF. El tráfico viaja cifrado de punta a punta: Navegador → Cloudflare (HTTPS) → balanceador (HTTPS con certificado de ACM) en modo **Full (strict)**.

Convención
En este documento **`<DOMINIO>`** representa el dominio registrado por el cliente (por ejemplo, `ejemplo.com`); debe reemplazarse en todos los comandos y ejemplos. La implementación de referencia usa `<DOMINIO>`.

10.1 Configuración inicial (una sola vez)

1. Registrar el dominio en el registrador de preferencia (por ejemplo, Nameslink, Namecheap o Cloudflare Registrar).
2. Agregarlo a Cloudflare (plan Free) y copiar los dos nameservers asignados.
3. En el registrador, **reemplazar** sus nameservers por los de Cloudflare. Cuando el dominio aparece como **Active**, queda listo.

10.2 Pasos por cada cuenta de AWS

Los certificados de ACM **no se transfieren entre cuentas de AWS**; con una cuenta nueva se solicita uno nuevo, y el DNS del balanceador es distinto en cada pila. En los ejemplos, `<DOMINIO>` es el dominio registrado por el cliente (por ejemplo, `ejemplo.com`).

Paso	Comando	Resultado
1	`./scripts/desplegar.sh certificado <DOMINIO>`	Solicita el certificado y muestra el CNAME de validación (con token de Cloudflare lo crea automáticamente). Espera hasta que quede **emitido**.
2	`./scripts/desplegar.sh up`	Activa HTTPS en el balanceador (listener 443) y la cookie `Secure`.
3	`./scripts/desplegar.sh cloudflare <DOMINIO>`	Apunta el dominio al balanceador y aplica los ajustes de la zona. Sin token, `dominio` imprime los registros para crearlos manualmente.

Ejemplo con token de Cloudflare (flujo automático). Primero se carga el token sin que quede en el historial de la terminal:

```
read -rs CLOUDFLARE_API_TOKEN && export CLOUDFLARE_API_TOKEN   # pegar el token (no se muestra) y pulsar Enter
```

Paso 1 — certificado. El comando crea el registro de validación en Cloudflare y espera la emisión (2 a 10 minutos):

```
./scripts/desplegar.sh certificado <DOMINIO>

==> Certificado de ACM para <DOMINIO> (validación por DNS)
  ✓ solicitud creada
==> Creando el registro de validación en Cloudflare con su token (automático)
  ✓ token de Cloudflare válido; zona <DOMINIO> (0123456789abcdef0123456789abcdef)
  ✓ DNS creado: _a3d6….<DOMINIO> → _c902….acm-validations.aws  (proxy: false)
==> Esperando la validación (2-10 min ...)
  ✓ certificado EMITIDO: arn:aws:acm:us-east-1:123456789012:certificate/…
```

Paso 2 — activar HTTPS en el balanceador (si ya hay una pila desplegada se usa `--sin-build` para no reconstruir las imágenes):

```
./scripts/desplegar.sh up --sin-build
  ✓ HTTPS activado con el certificado de ACM (cookie Secure: true)
```

Paso 3 — apuntar el dominio al balanceador y aplicar los ajustes de la zona:

```
./scripts/desplegar.sh cloudflare <DOMINIO>

==> Configurando Cloudflare para <DOMINIO> mediante su API
  ✓ token de Cloudflare válido; zona <DOMINIO> (0123456789abcdef0123456789abcdef)
  ✓ certificado ya emitido: no hace falta crear el registro de validación
  ✓ DNS actualizado: <DOMINIO> → <balanceador>.us-east-1.elb.amazonaws.com  (proxy: true)
==> Ajustes de seguridad de la zona
  ✓ ajuste ssl = strict
  ✓ ajuste always_use_https = on
  ✓ ajuste min_tls_version = 1.2
  ✓ ajuste rocket_loader = off
==> Estado actual
```

```
  ✓ https://<DOMINIO>/salud responde 200: el dominio ya funciona
```

Ejemplo sin token (configuración manual guiada). `dominio` imprime exactamente qué crear en el panel de Cloudflare (*DNS → Records → Add record*) y comprueba el estado:

```
./scripts/desplegar.sh dominio <DOMINIO>

 REGISTRO 1 — validación del certificado de ACM   estado: ISSUED
   Tipo: CNAME   Nombre: _a3d6….<DOMINIO>
   Destino: _c902….acm-validations.aws   Proxy status: DNS only (nube GRIS)
 REGISTRO 2 — el sitio apunta al balanceador
   Tipo: CNAME   Nombre: @ (la raíz)   Proxy status: Proxied (nube NARANJA)
   Destino: <balanceador>.us-east-1.elb.amazonaws.com
 AJUSTE 3 — SSL/TLS → Overview: modo "Full (strict)"
```

Si el dominio es un subdominio (por ejemplo `menu.<DOMINIO>`), el nombre del registro 2 es `menu` en lugar de `@`. Si el comando indica que ya existe un registro de otro tipo con el mismo nombre, se elimina en el panel o se vuelve a ejecutar `cloudflare` con `--reemplazar`.

10.3 Configuración que aplica el comando cloudflare

Acción	Detalle
Registro de validación del certificado	CNAME hacia ACM, *DNS only* (nube gris); solo si el certificado aún no está emitido.
CNAME del sitio	`<DOMINIO>` → DNS del balanceador, *Proxied* (nube naranja). Si ya existe, lo **actualiza** (el DNS del balanceador cambia con cada pila nueva).
Ajustes de la zona	SSL/TLS **Full (strict)**, *Always Use HTTPS*, TLS mínimo **1.2**, *Rocket Loader* desactivado (inyecta scripts, incompatibles con la política CSP).
Verificación	Consulta `https://<DOMINIO>/salud` e interpreta el resultado (200, 522, 525…).

El comando es **idempotente**: puede repetirse sin efectos secundarios. Para dominios de sufijo compuesto (`midominio.co.uk`) se indica la zona con la variable `PNK_CF_ZONA`, por ejemplo `PNK_CF_ZONA=midominio.co.uk ./scripts/desplegar.sh cloudflare midominio.co.uk`.

10.4 Token de la API de Cloudflare

El token es la credencial que permite al script modificar el DNS y los ajustes de la zona. Se crea una sola vez desde el panel de Cloudflare:

1. Iniciar sesión en https://dash.cloudflare.com, abrir el menú del perfil (arriba a la derecha) → **My Profile → API Tokens → Create Token**.
2. Elegir **Create Custom Token** (*Get started*) y ponerle un nombre, por ejemplo `cartapnk-despliegue`.
3. En **Permissions** agregar tres filas: `Zone` · `DNS` · `Edit`; `Zone` · `Zone Settings` · `Edit`; y `Zone` · `Zone` · `Read`.
4. En **Zone Resources** elegir `Include` · `Specific zone` · **el dominio del cliente** (así el token no puede operar sobre otras zonas).
5. *(Recomendado)* En **TTL** fijar una fecha de vencimiento y, si la IP de despliegue es conocida, restringirlo en **Client IP Address Filtering**.
6. **Continue to summary → Create Token.** Copiar el valor que se muestra: **aparece una sola vez**. Guardarlo en un gestor de contraseñas.

Para usarlo se carga como variable de entorno sin escribirlo en el historial de la terminal (el propio comando valida el token al iniciar y falla con un mensaje claro si es inválido, venció o no ve la zona):

```
read -rs CLOUDFLARE_API_TOKEN && export CLOUDFLARE_API_TOKEN
```

Seguridad del token

Es una credencial: permite modificar el DNS del dominio. Por eso se limita a **una zona**, con los **permisos mínimos** y vencimiento.

Se transmite solo por variable de entorno y por la entrada estándar de `curl` (`-K -`): **no aparece en la lista de procesos ni en el historial** y nunca se escribe en el repositorio.

Ante una filtración: Cloudflare → API Tokens → **Roll** (cambia el valor) o **Delete**.

10.5 Diagnóstico

Síntoma	Causa probable	Acción
Error **522 / 523 / 524**	El CNAME apunta a un balanceador que ya no existe o no responde	`./scripts/desplegar.sh cloudflare` (o editar el destino con el DNS actual)
Error **525 / 526**	SSL de Cloudflare no está en Full (strict) o el balanceador aún no usa el certificado	Revisar el modo SSL y ejecutar `up --sin-build`
El dominio no responde	El DNS aún no se propaga, o el sitio no está desplegado	Esperar unos minutos y ejecutar `./scripts/desplegar.sh dominio`
"Esta cuenta de AWS no tiene un certificado…"	Cuenta de AWS distinta de la que emitió el certificado	`./scripts/desplegar.sh certificado <DOMINIO>`

11. Operación y recuperación

11.1 Escenarios de pérdida de recursos

Si se elimina…	Efecto	Acción
Una instancia EC2	Sin impacto visible: el Auto Scaling crea otra en 3 a 5 minutos y la restante atiende mientras tanto. Sesiones y datos no dependen de la instancia.	Ninguna.
Todas las instancias	El sitio se interrumpe unos minutos y el grupo las recrea automáticamente.	Esperar; verificar con `status`.
El equipo desde donde se ejecuta el script (CloudShell, servidor de trabajo)	La producción no se afecta. Se pierde la carpeta `.despliegue/` (secretos locales).	Clonar de nuevo el repositorio.
La pila de CloudFormation	Se eliminan red, balanceador, instancias, **RDS (con sus datos)** y **el bucket S3 (con sus imágenes)**. El sitio queda fuera de servicio.	Reinstalar (sección 11.2).
El repositorio ECR	La infraestructura sigue operando, pero no podrá lanzar instancias nuevas.	`up` reconstruye y publica las imágenes.

Si se elimina…	Efecto	Acción
El certificado de ACM	HTTPS deja de funcionar al renovar o recrear el balanceador.	`certificado <DOMINIO>` y luego `up --sin-build`.
El registro DNS en Cloudflare	El dominio deja de resolver.	`cloudflare` (o recrear el CNAME).

Se restablece al reinstalar: el código (GitHub), el catálogo inicial y el usuario administrador. **No se restablece:** el contenido creado desde el panel (restaurantes, platillos e imágenes) si no existen copias de seguridad; ver recomendaciones en la sección 13.

11.2 Reinstalación completa (~25 minutos)

Es válida también para una cuenta de AWS distinta.

1. Configurar las credenciales de AWS en `~/.aws/credentials` y la región (`aws configure set region us-east-1`); `aws sts get-caller-identity` debe mostrar la cuenta.
2. `git clone https://github.com/Alkr4/CartaPnk.git && cd CartaPnk`
3. `./scripts/desplegar.sh verificar`
4. `./scripts/desplegar.sh certificado <DOMINIO>` (reutiliza el certificado si existe; en una cuenta nueva solicita uno y crea su registro de validación).
5. `./scripts/desplegar.sh up`
6. `./scripts/desplegar.sh cloudflare <DOMINIO>` (o `dominio` para crear los registros manualmente).
7. Probar `https://<DOMINIO>` y `https://<DOMINIO>/admin`. La contraseña del administrador está en `.despliegue/secretos.env`.

11.3 Gestión de secretos

`up` genera contraseñas aleatorias (base de datos, sesión y administrador) y las guarda en `.despliegue/secretos.env` del equipo desde el que se ejecuta; no se versionan en Git. **Se recomienda almacenarlas en un gestor de contraseñas.** Si se pierde el archivo y la pila sigue activa, un nuevo `up` genera secretos nuevos y actualiza la pila: la contraseña del administrador cambia y no hay pérdida de datos.

11.4 Solución de problemas

Síntoma	Causa probable	Acción
`ExpiredToken` / "credenciales no válidas"	Credenciales temporales vencidas	Renovar las credenciales
`verificar` marca ✗ en algún servicio	El servicio no está habilitado en la cuenta o la sesión caducó	Habilitar el servicio o renovar la sesión y repetir
La pila falla por versión del motor o clase de RDS	PostgreSQL 17 o `db.t3.micro` no disponibles en la región	`PNK_DB_ENGINE_VERSION=16 ./scripts/desplegar.sh up`
AccessDenied en el perfil de instancia	El perfil indicado no existe con ese nombre	`PNK_INSTANCE_PROFILE=<nombre> ./scripts/desplegar.sh up`

Síntoma	Causa probable	Acción
"Aún no hay instancias saludables"	La instancia sigue arrancando o no pudo descargar las imágenes	`status`; revisar `aws ec2 get-console-output` y los mensajes de `pnk-userdata`
`docker build` se detiene en `npm ci`	Memoria insuficiente (servidor de 1 GB) sin swap	Crear un swap de 2 GB o usar un equipo con más memoria

12. Pruebas y calidad

Capa	Herramienta	Qué verifica
Backend (136 pruebas, 7 conjuntos)	Jest + Supertest contra PostgreSQL real	Datos públicos de la carta; carrito (el precio lo define la base de datos); CSRF; cabeceras y cookies; control de acceso por rol; fuerza bruta (429); CRUD completo; validación (422) y asignación masiva; subidas hostiles (SVG, script, ejecutable, > 2 MB); `/media` sin recorrido de rutas; almacenamiento local y S3.
Frontend (16 pruebas)	`node:test` + jsdom (sin base de datos)	Renderizado de la carta, el carrito y el panel; **anti-XSS** (el HTML de la API se muestra como texto); URL de imagen no internas; envío del token CSRF; errores 422 por campo; confirmación antes de eliminar; ausencia de scripts en línea; configuración de Nginx.
Integración front-back	Páginas reales (jsdom) contra la API real	Acceso, alta de restaurante y carta, validación 422, eliminación y carrito, de extremo a extremo.
Análisis estático	ESLint + `eslint-plugin-security` (`--max-warnings 0`)	Inyección, expresiones regulares inseguras, acceso dinámico a objetos, `eval`.
Dependencias	`npm audit --omit=dev`	Sin vulnerabilidades en las dependencias de producción.
Infraestructura	`cfn-lint` y ShellCheck (informativo)	Plantilla de CloudFormation y scripts.

Además de las pruebas automatizadas, el ambiente QA y el despliegue en AWS se validan con comprobaciones manuales: salud a través de Nginx hasta la base de datos, cabeceras de seguridad, `401` en la API del panel sin sesión y `404` en rutas desconocidas.

13. Consideraciones para producción y hoja de ruta

La configuración actual está dimensionada para un entorno de evaluación y demostración. Para una operación de producción con requisitos de continuidad del negocio se recomiendan las siguientes mejoras:

Área	Recomendación
Datos	Habilitar copias de seguridad automáticas y **Multi-AZ** en RDS (`BackupRetentionPeriod`), y **versionado** del bucket S3. Hoy la eliminación de la pila elimina los datos.

Área	Recomendación
Red	Mover las instancias a subredes privadas con NAT Gateway (o VPC endpoints para ECR y S3).
Secretos	Gestionar los secretos con AWS Secrets Manager (`ManageMasterUserPassword` en RDS) en lugar de pasarlos por el `user-data` de la plantilla de lanzamiento.
Permisos	Reemplazar el perfil de instancia amplio por un rol de mínimo privilegio con acceso solo al bucket y al repositorio ECR.
Borde	Activar las capas 5 a 7 de la sección 6 según el modelo de acceso del equipo de administración, y AWS WAF si se requiere inspección adicional.
CI/CD	Poner en marcha el pipeline de Jenkins de la sección 8.3 y activar la protección de la rama `main`.
Observabilidad	Centralizar los registros JSON (CloudWatch Logs) y definir alarmas sobre `/salud`, CPU y errores 5xx del balanceador.
Funcionalidad	Si se requiere que los clientes comenten, añadir comentarios anónimos con moderación previa (hoy el acceso es exclusivo del personal autorizado).